MECHATRONICS AND IOT

ASSIGNMENT REPORT – 13

TITLE:

Design and develop a Water Level Detection and Flood Alert System using Ultrasonic Sensor.The system measures water levels in rivers, tanks, or dams and provides early warning alerts during overflow conditions.

TEAM MEMBERS:

HARISANKARAN S

(24MER013)

MUTHUKUMAR M

(24MER034)

MONESHWARAN C

(24MER032)

PROJECT OVERVIEW

The **Water Level Detection and Flood Alert System** is designed to continuously monitor the water level in rivers, tanks, dams, and other water bodies. The system uses an **ultrasonic sensor** to measure the distance between the sensor and the water surface.The measured water level is processed by a microcontroller. When the water level reaches a predefined warning or critical level, the system provides an alert through a **buzzer, LED, display, or other warning device**.The system helps provide early warnings during rising water levels and overflow conditions.

PROBLEM STATEMENT

- Manual monitoring of water levels.
- Delay in detecting rapidly rising water levels.
- Difficulty in continuously monitoring rivers, tanks, and dams.
- Lack of automatic flood warning systems.
- Risk of overflow and flooding.
- Lack of real-time water-level information.
- Need for early warning during critical water levels.
- Human involvement required for regular inspection.

PROPOSED SOLUTION

The proposed system uses an **ultrasonic sensor and microcontroller** to automatically monitor the water level.The ultrasonic sensor is positioned above the water surface. It sends ultrasonic waves toward the water surface and receives the reflected signal. The microcontroller calculates the distance and determines the corresponding water level.

The system can provide:

- Automatic water-level measurement.
- Continuous monitoring.
- Warning-level detection.
- Critical-level detection.
- Buzzer alert.
- LED indication.
- Water-level display.
- Early flood/overflow warning.

When the water reaches the critical level, the system activates the warning devices to alert nearby people.

WORKING PRINCIPLE

- The ultrasonic sensor transmits an ultrasonic pulse toward the water surface.
- The pulse is reflected from the water surface.
- The sensor receives the reflected echo.
- The microcontroller measures the time taken for the echo to return.
- The distance between the sensor and water surface is calculated.
- The water level is determined based on the measured distance.
- The measured level is compared with predefined threshold values.
- If the water level reaches the warning level, a warning indication is activated.
- If the water reaches the critical level, the buzzer/alert system is activated.
- The system continuously repeats the measurement process.

SYSTEM ARCHITECTURE

Ultrasonic Sensor → Microcontroller → Water-Level Calculation → Threshold Comparison → Display/LED/Buzzer Alert

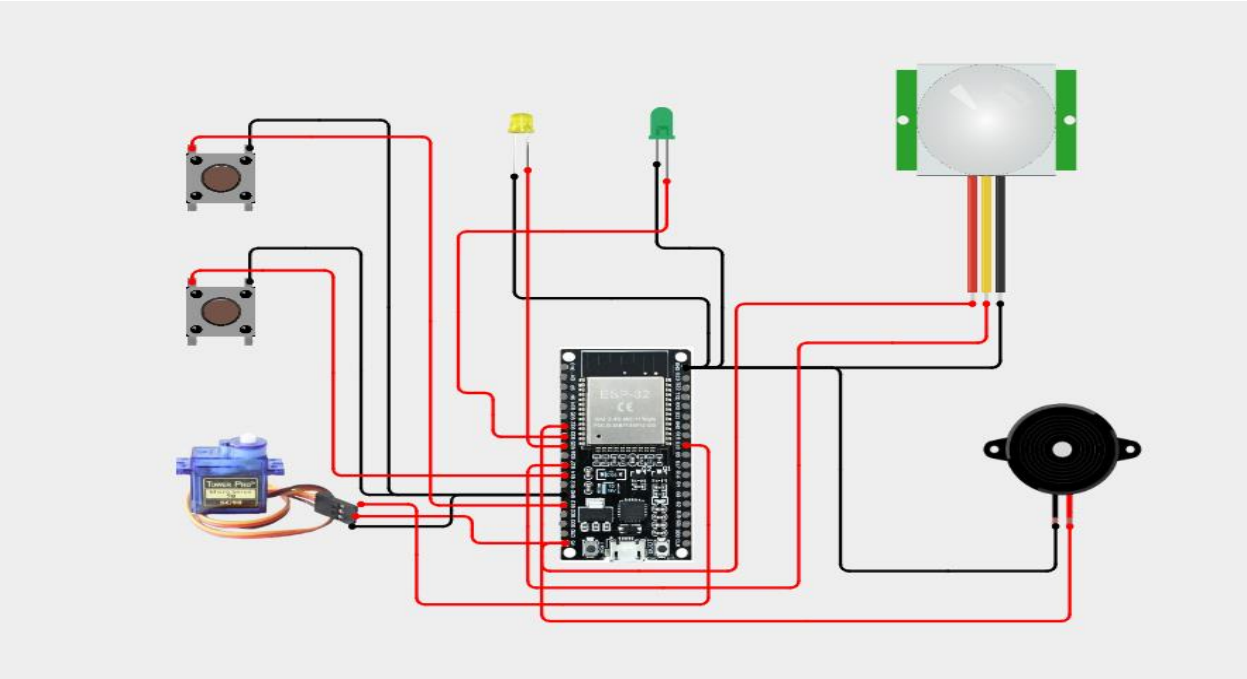
WATER LEVEL OPERATING CONDITIONS

Parameter	Normal	Warning	Critical
Water Level	Safe level	Rising/high level	Overflow/critical level
Ultrasonic Distance	Large distance	Reduced distance	Minimum distance
Flood Condition	No flood risk	Possible flood risk	High flood risk
Buzzer	OFF	Intermittent/Warning	ON
LED	Normal indication	Warning indication	Critical indication
System Status	Normal	Alert	Emergency

PARAMETERS MONITORED

Parameter	Sensor/Device	Purpose
Water Level	Ultrasonic Sensor	Measure water level
Sensor Distance	Ultrasonic Sensor	Determine distance to water surface
Warning Level	Microcontroller	Detect high water level
Critical Level	Microcontroller	Detect overflow condition
System Status	LED/Display	Show current condition
Flood Alert	Buzzer	Provide warning indication

CIRCUIT DESIGN



ALGORITHM

- Initialize the microcontroller.
- Initialize the ultrasonic sensor.
- Initialize the display.
- Initialize LEDs and buzzer.
- Set the safe, warning, and critical water-level thresholds.
- Trigger the ultrasonic sensor.
- Measure the echo time.
- Calculate the distance to the water surface.
- Calculate the water level.
- Display the measured water level.
- Compare the water level with the threshold.
- If the water level is normal, keep the alarm OFF.
- If the water level reaches the warning level, activate the warning indication.
- If the water level reaches the critical level, activate the buzzer and critical alert.
- Continue monitoring the water level.
- Repeat the process.Allow the passenger to exit.
- Check for human presence again.
- Return to standby mode when no passenger is detected.
- **STOP/REPEAT**.

CODING

```
#include <WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ================================================
// WIFI DETAILS
// ================================================

#define WIFI_SSID "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD "qkts5737"

// ================================================
// ADAFRUIT IO DETAILS
// ================================================

#define IO_USERNAME "Hair_13"
#define IO_KEY "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASSWORD);

// ================================================
// ADAFRUIT IO FEEDS
// ================================================

AdafruitIO_Feed *waterDistanceFeed = io.feed("water-distance");
AdafruitIO_Feed *waterLevelFeed = io.feed("water-level");
AdafruitIO_Feed *floodStatusFeed = io.feed("flood-status");

// ================================================
// PIN CONNECTIONS
// ================================================

// Ultrasonic sensor
const int TRIG_PIN = 5;
const int ECHO_PIN = 18;

// LEDs
```

```
const int GREEN_LED = 25;
const int RED_LED = 26;

// Buzzer
const int BUZZER = 27;

// LCD
const int SDA_PIN = 21;
const int SCL_PIN = 22;

LiquidCrystal_I2C lcd(0x27, 16, 2);

// ================================================
// FLOOD LEVEL SETTINGS
// ================================================

const float SAFE_DISTANCE = 30.0;
const float DANGER_DISTANCE = 10.0;

// ================================================
// FUNCTION DECLARATIONS
// ================================================

float getDistance();
int calculateWaterLevel(float distance);
void displayWaterStatus(float distance, const String &status);
void setIndicators(bool green, bool red, bool buzzer);

// ================================================
// SETUP
// ================================================

void setup()
{
  Serial.begin(115200);

  // Ultrasonic sensor
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
```

```
// LEDs
pinMode(GREEN_LED, OUTPUT);
pinMode(RED_LED, OUTPUT);

// Buzzer
pinMode(BUZZER, OUTPUT);

// LCD
Wire.begin(SDA_PIN, SCL_PIN);
lcd.init();
lcd.backlight();

// Startup display
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("SMART FLOOD");
lcd.setCursor(0, 1);
lcd.print("MONITORING");

delay(2000);
lcd.clear();

// ================================================
// CONNECT TO ADAFRUIT IO
// ================================================

Serial.println();
Serial.println("Connecting to Adafruit IO...");

io.connect();

while (io.status() < AIO_CONNECTED)
{
  Serial.print(".");
  delay(500);
}

Serial.println();
Serial.println("Adafruit IO Connected!");
```

```
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Adafruit IO");
  lcd.setCursor(0, 1);
  lcd.print("Connected!");

  delay(2000);
  lcd.clear();
}

// ================================================
// GET ULTRASONIC DISTANCE
// ================================================

float getDistance()
{
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);

  digitalWrite(TRIG_PIN, LOW);

  long duration = pulseIn(ECHO_PIN, HIGH, 30000);

  if (duration == 0)
  {
    return -1;
  }

  float distance = duration * 0.0343 / 2.0;

  return distance;
}

// ================================================
// CALCULATE WATER LEVEL
// ================================================
```

```
int calculateWaterLevel(float distance)
{
  int level = ((SAFE_DISTANCE - distance) / SAFE_DISTANCE) * 100;

  if (level < 0)
  {
    level = 0;
  }

  if (level > 100)
  {
    level = 100;
  }

  return level;
}

// ================================================
// SET LED AND BUZZER
// ================================================

void setIndicators(bool green, bool red, bool buzzer)
{
  digitalWrite(GREEN_LED, green ? HIGH : LOW);
  digitalWrite(RED_LED, red ? HIGH : LOW);
  digitalWrite(BUZZER, buzzer ? HIGH : LOW);
}

// ================================================
// DISPLAY WATER STATUS
// ================================================

void displayWaterStatus(float distance, const String &status)
{
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("Water:");
  lcd.print(distance, 1);
  lcd.print("cm");
```

```
  lcd.setCursor(0, 1);

  if (status == "SAFE")
  {
    lcd.print("STATUS: SAFE");
  }
  else if (status == "WARNING")
  {
    lcd.print("WARNING!");
  }
  else
  {
    lcd.print("!!! FLOOD !!!");
  }
}

// ================================================
// MAIN LOOP
// ================================================

void loop()
{
  // Keep Adafruit IO connected
  io.run();

  // Read ultrasonic sensor
  float distance = getDistance();

  // ================================================
  // SENSOR ERROR
  // ================================================

  if (distance < 0)
  {
    Serial.println("Ultrasonic sensor error!");

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Sensor Error!");
```

```
  lcd.setCursor(0, 1);
  lcd.print("Check HC-SR04");

  setIndicators(false, false, false);

  delay(2000);

  return;
}

// ================================================
// CALCULATE WATER LEVEL
// ================================================

int waterLevel = calculateWaterLevel(distance);

String floodStatus;

// ================================================
// SAFE CONDITION
// ================================================

if (distance > SAFE_DISTANCE)
{
  floodStatus = "SAFE";

  setIndicators(true, false, false);

  displayWaterStatus(distance, floodStatus);
}

// ================================================
// WARNING CONDITION
// ================================================

else if (distance > DANGER_DISTANCE &&
         distance <= SAFE_DISTANCE)
{
  floodStatus = "WARNING";
```

```
  setIndicators(false, true, true);

  delay(200);

  digitalWrite(BUZZER, LOW);

  displayWaterStatus(distance, floodStatus);
}

// ================================================
// FLOOD CONDITION
// ================================================

else
{
  floodStatus = "FLOOD";

  setIndicators(false, true, true);

  displayWaterStatus(distance, floodStatus);
}

// ================================================
// SERIAL MONITOR
// ================================================

Serial.println("----------------------------");

Serial.print("Distance: ");
Serial.print(distance);
Serial.println(" cm");

Serial.print("Water Level: ");
Serial.print(waterLevel);
Serial.println(" %");

Serial.print("Status: ");
Serial.println(floodStatus);

// ================================================
```

```
  // SEND DATA TO ADAFRUIT IO
  // ================================================

  waterDistanceFeed->save(distance);
  waterLevelFeed->save(waterLevel);
  floodStatusFeed->save(floodStatus.c_str());

  Serial.println("Data sent to Adafruit IO!");
  Serial.println("---------------------------");

  // Wait before next reading
  delay(5000);
}
```

SYSTEM WORKING

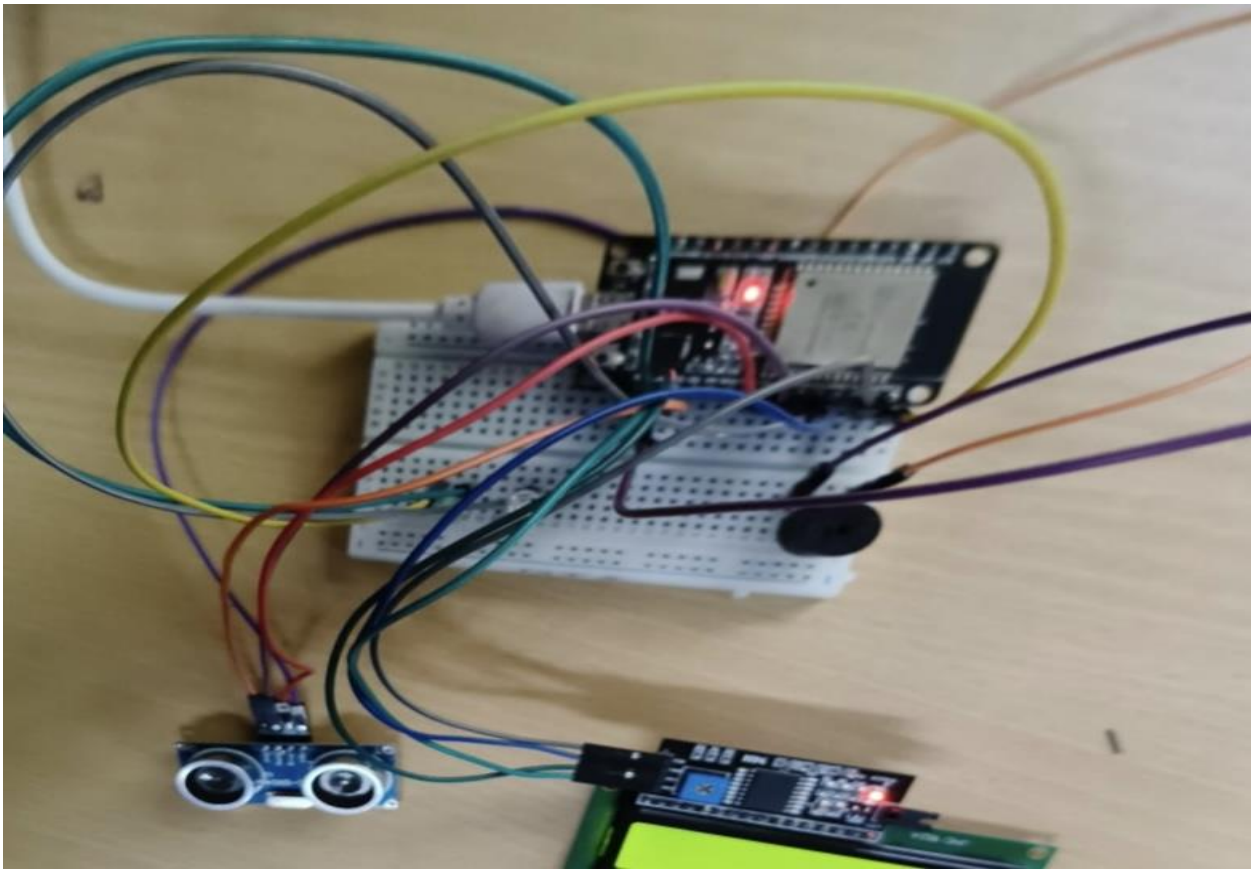
The ultrasonic sensor is mounted above the water surface. It continuously sends ultrasonic waves toward the water.When the water level rises, the distance between the sensor and water surface decreases. The sensor detects this change and sends the measurement to the microcontroller.The microcontroller calculates the water level and compares it with predefined threshold values.Under normal conditions, the system indicates a safe water level. When the water level rises above the warning threshold, the warning LED or alert is activated.When the water reaches the critical level, the buzzer is activated to provide an early flood/overflow warning.After the water level decreases below the critical threshold, the system can return to the normal monitoring condition.

COMPONENTS

S.No.	Component	Quantity	Purpose
1	Arduino/ESP32 Microcontroller	1	Main controller
2	Ultrasonic Sensor	1	Water-level measurement
3	Buzzer	1	Flood warning
4	LEDs	As required	Status indication
5	LCD/OLED Display	1	Display water level
6	Resistors	As required	Circuit protection
7	Breadboard	1	Circuit assembly
8	Jumper Wires	1 set	Circuit connections
9	Power Supply	1	Power source

PROTOTYPE IMPLEMENTATION

The prototype can be implemented using an **Arduino/ESP32 microcontroller, ultrasonic sensor, buzzer, LEDs, and display**.The ultrasonic sensor is mounted at a suitable height above the water container. The sensor is connected to the microcontroller.The microcontroller receives the echo signal from the ultrasonic sensor and calculates the distance between the sensor and water surface. This information is converted into the corresponding water level.An LCD/OLED display can be used to show the water level and system condition. LEDs can indicate normal, warning, and critical conditions, while a buzzer provides an audible flood alert.The system can be simulated using platforms such as **Proteus or Tinkercad**, depending on the required implementation.



RESULTS & PERFORMANCE ANALYSIS

Condition	Water-Level Status	System Action
Low/Normal water	Safe	Normal indication
Water rising	Warning	Warning LED activated
High water level	Critical	Buzzer/critical alert activated
Overflow condition	Emergency	Flood warning provided
Water level decreases	Normal	System returns to monitoring

PERFORMANCE

- **Automatic water-level detection**
- **Continuous water-level monitoring**
- **Real-time measurement**
- **Early flood warning**
- **Automatic overflow detection**
- **Warning and critical-level indication**
- **Buzzer-based alert system**
- **LED status indication**
- **Water-level display**
- **Reduced dependence on manual monitoring**
- **Improved flood-awareness and safety**
- **Low-cost prototype suitable for educational and simulation applications**